BONUS MODULE

Fragments in Depth

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Many Android apps in companies, including big parts of the app your team probably works on, are built with **Activities, Fragments and XML layouts**. New screens are often written in Compose, but you'll read, fix and extend fragment code almost every week. This module teaches fragments from zero to team level, with real code.

Day	Topic	You will be able to...
Day 1	What is a fragment?	Create a fragment with a layout and show it in an activity
Day 2	The fragment lifecycle	Know which callback runs when, and the two lifecycles (fragment and view)
Day 3	View Binding in fragments	Access views safely and avoid the most common memory leak
Day 4	FragmentManager, arguments and results	Replace fragments, use the back stack, pass data in and out
Day 5	ViewModel, Flow and RecyclerView	Build a real list screen the way teams do it
Day 6	Navigation, dialogs and Compose in fragments	Use the Navigation component, Safe Args, dialogs and ComposeView

Before you start

- Create a practice project: **New Project** → **Empty Views Activity** (not "Empty Activity", which is Compose).
- To keep the code short, **import lines are left out**. When a word is red in Android Studio, click it and press **Alt+Enter** (Option+Enter on Mac) → Import.
- Compare every pattern with your team's code. You'll see the same ones again and again.

> **Tip:** The Kotlin code in this module was **compile-checked** with the Kotlin and Compose compilers against stand-ins that copy the Android fragment, lifecycle, RecyclerView and Navigation APIs (the real Android libraries couldn't be downloaded here). XML layouts and Gradle files are shown as they are. Run every example on an emulator or phone.

BONUS MODULE • DAY 1

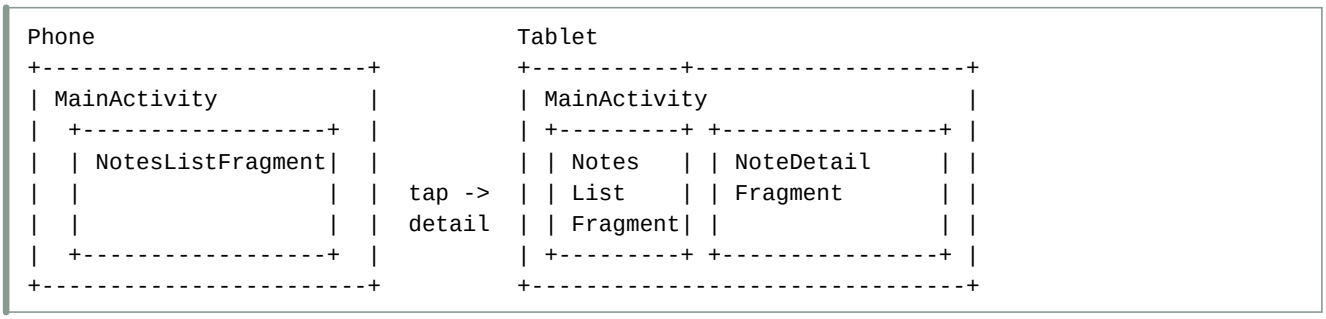
What Is a Fragment?

Today's goal: Understand what fragments are and why apps use them, then create a fragment and show it inside an activity.

1. The idea

A **Fragment** is a reusable piece of a screen with its own layout and its own code. It always lives **inside an Activity** (its **host**). One activity can show one fragment, or several at once, and can swap them as the user moves around the app.

Sketch: One activity, different fragments



2. Why fragments exist

Reason	What it means
Reuse	The same fragment can appear in different activities or layouts
Single-activity apps	Most modern View-based apps have ONE activity and many fragments, one per screen
Big screens	On a tablet, two fragments can sit side by side
Navigation and back stack	Fragments can be swapped with animations; Back returns to the previous one
Smaller classes	Each screen's code lives in its own fragment instead of one huge activity

	Activity	Fragment	Composable screen
What it is	A whole window, an entry point	A piece of UI inside an activity	A function that draws UI
In the manifest?	Yes	No	No
Layout	XML or Compose	XML (or ComposeView)	Compose
Where you'll see it	Every app (at least one)	Most existing team apps	New screens

3. Setup

app/build.gradle.kts (Android only, versions as in your project)

```
android {
    buildFeatures {
        viewBinding = true          // used from Day 3
    }
}

dependencies {
    implementation("androidx.fragment:fragment-ktx:1.8.5")      // Fragment + Kotlin helpers
    implementation("androidx.appcompat:appcompat:1.7.0")
}
```

4. Your first fragment: the layout

res/layout/fragment_hello.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/greeting"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello from a fragment"
        android:textSize="22sp" />

    <Button
        android:id="@+id/countButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="Tap me" />
</LinearLayout>
```

5. Your first fragment: the class

HelloFragment.kt

```
class HelloFragment : Fragment(R.layout.fragment_hello) {     // pass the layout to the constructor

    private var count = 0

    // Called when the fragment's view is ready. Set up views and click listeners here.
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val greeting = view.findViewById<TextView>(R.id.greeting)
        val button = view.findViewById<Button>(R.id.countButton)

        button.setOnClickListener {
            count++
            greeting.text = "Tapped $count times"
        }
    }
}
```

Piece	Meaning
Fragment(R.layout.fragment_hello)	Android inflates (creates) this layout for you as the fragment's view
onViewCreated(view, ...)	The view exists now: find views, set text, add listeners
view.findViewById<TextView>(...)	Finds a view by its id inside this fragment's layout (Day 3 shows a better way)

6. Showing the fragment in an activity (way 1: in XML)

The activity layout gets a **FragmentContainerView**: a box where fragments are shown. With **android:name**, the fragment is added automatically when the activity starts.

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragmentContainer"
    android:name="com.anjana.fragments.HelloFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

MainActivity.kt

```
class MainActivity : AppCompatActivity(R.layout.activity_main)      // that's all
```

7. Showing the fragment from code (way 2)

Without android:name, the activity adds the fragment itself with a **transaction**. This is the way you'll use when the activity decides which fragment to show.

MainActivity.kt, adding the fragment in code

```
class MainActivity2 : AppCompatActivity(R.layout.activity_main) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        if (savedInstanceState == null) {             // only the first time!
            supportFragmentManager.commit {
                setReorderingAllowed(true)
                add<HelloFragment>(R.id.fragmentContainer)
            }
        }
    }
}
```

Careful: Why **if (savedInstanceState == null)**? When the phone rotates, Android recreates the activity and **restores the fragments by itself**. Without the check, every rotation would add one more HelloFragment on top of the old one. This is one of the most common fragment bugs.

Tip: Notice that **count** resets to 0 when you rotate the phone, because the fragment object is recreated. On Day 5 the count moves into a **ViewModel**, which survives rotation.

Practice Exercises

1. Create **ProfileFragment** with a layout that shows your name and a "Follow" button. When tapped, the button text changes to "Following". Show it from MainActivity in code.

2. Add an **EditText** and a Button to HelloFragment. When the button is tapped, show "Hello, <name>!" in the TextView, or a **Toast** saying "Enter your name" if the field is empty.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Can a fragment exist without an activity?

Q2. What is FragmentContainerView?

Q3. Why check savedInstanceState == null before adding a fragment in onCreate?

Q4. Which callback is the right place to set click listeners in a fragment?

BONUS MODULE • DAY 2

The Fragment Lifecycle

Today's goal: Learn every lifecycle callback, the difference between the fragment's lifecycle and its view's lifecycle, and what happens in common situations.

1. The callbacks in order

Sketch: Fragment lifecycle

```
     onAttach()          fragment is connected to its activity
     onCreate()          fragment object created (no view yet)
  +-> onCreateView()     create the view (done for you with Fragment(R.layout.x))
  |   onViewCreated()    view ready: set up views here
  |   onStart()          visible
  |   onResume()         in front, user can interact
  |       ... user uses the screen ...
  |   onPause()          losing focus
  |   onStop()           not visible
  +-- onDestroyView()    view destroyed (fragment may still exist!)
     onDestroy()         fragment object destroyed
     onDetach()          disconnected from the activity
```

Callback	Use it for	Don't...
onCreate	Read arguments, set up things that don't need the view	Touch views (they don't exist yet)
onViewCreated	Find views/binding, set listeners, start collecting data for the UI	Forget super.onViewCreated
onStart / onStop	Start/stop things that should only run while visible	
onResume / onPause	Rarely needed: camera, sensors, things that need focus	Save important data only here
onDestroyView	Clear references to views (binding = null, Day 3)	Keep views in fields after this

2. See it yourself: log every callback

LifecycleFragment.kt

```
class LifecycleFragment : Fragment(R.layout.fragment_hello) {
    private val logTag = "Lifecycle"

    override fun onAttach(context: Context) { super.onAttach(context); Log.d(logTag, "onAttach") }
    override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState); Log.d(logTag, "onCreate") }
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState); Log.d(logTag, "onViewCreated")
    }
    override fun onStart() { super.onStart(); Log.d(logTag, "onStart") }
    override fun onResume() { super.onResume(); Log.d(logTag, "onResume") }
    override fun onPause() { super.onPause(); Log.d(logTag, "onPause") }
    override fun onStop() { super.onStop(); Log.d(logTag, "onStop") }
    override fun onDestroyView() { super.onDestroyView(); Log.d(logTag, "onDestroyView") }
    override fun onDestroy() { super.onDestroy(); Log.d(logTag, "onDestroy") }
    override fun onDetach() { super.onDetach(); Log.d(logTag, "onDetach") }
}
```

Run it, open **Logcat**, type **tag:Lifecycle** in the filter, and try each situation below. Writing down what you see is the best way to remember the lifecycle.

What you do	What Logcat shows
Open the app	onAttach, onCreate, onViewCreated, onStart, onResume
Press Home	onPause, onStop
Come back to the app	onStart, onResume
Rotate the phone	onPause, onStop, onDestroyView, onDestroy, onDetach, then the full start again: onAttach ... onResume
Replace it with another fragment using addToBackStack (Day 4)	onPause, onStop, onDestroyView (NOT onDestroy: the fragment waits on the back stack)
Press Back to return to it	onViewCreated, onStart, onResume (a NEW view for the SAME fragment)
Press Back to leave the app	onPause, onStop, onDestroyView, onDestroy, onDetach

3. Two lifecycles: the fragment and its view

Look at the back stack rows above: the **view** was destroyed and created again, but the **fragment object** stayed alive. So a fragment has two lifecycles:

	Fragment lifecycle	View lifecycle
Starts	onCreate	onViewCreated
Ends	onDestroy	onDestroyView
Access it with	this (the fragment is a LifecycleOwner)	viewLifecycleOwner
Use it for	Things that don't touch views	Anything that updates views: observing data, collecting Flows

Careful: Anything that updates the UI must use **viewLifecycleOwner**, not the fragment itself. If you observe data with the fragment's lifecycle, then after going back and forth you get **two observers** (one for the old, dead view and one for the new view), and updates to the old view can crash. You'll use viewLifecycleOwner on Day 5.

4. requireContext(), requireActivity() and friends

Function	Returns	Crashes when...
requireContext()	The Context (non-null)	The fragment isn't attached (before onAttach or after onDetach)
requireActivity()	The host activity	Same as above
requireView()	The fragment's view	Called before onCreateView or after onDestroyView
context / activity / view	The same, but nullable	Never; they return null instead

Safe use inside callbacks

```
class ToastFragment : Fragment(R.layout.fragment_hello) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        view.findViewById<Button>(R.id.countButton).setOnClickListener {
            // Inside a click listener the fragment is surely attached, so require... is fine.
            Toast.makeText(requireContext(), "Clicked!", Toast.LENGTH_SHORT).show()
        }
    }
}
```

> **Tip:** "**IllegalStateException: Fragment not attached to a context**" means you called requireContext() too late, usually from a callback that finished after the user left the screen (an old network callback, a delayed Handler). Using viewLifecycleOwner.lifecycleScope for that work (Day 5) cancels it automatically.

Practice Exercises

1. Add the logging callbacks to your ProfileFragment from Day 1. Rotate the phone twice and write down the order. How many times did onCreate run?

2. Add **onSaveInstanceState(outState: Bundle)** to HelloFragment to save count, and restore it in onViewCreated from savedInstanceState, so the count survives rotation.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Which callback runs when a fragment's view is destroyed but the fragment stays on the back stack?

Q2. What is viewLifecycleOwner?

Q3. Why must UI updates use viewLifecycleOwner instead of the fragment?

Q4. What is the difference between requireContext() and context?

BONUS MODULE • DAY 3

View Binding in Fragments

Today's goal: Replace findViewById with View Binding, and use the team-standard pattern that avoids memory leaks.

1. What View Binding gives you

With **viewBinding = true** (Day 1 setup), Android Studio generates a **binding class** for every layout. **fragment_hello.xml** becomes **FragmentHelloBinding**, with a property for each view that has an id: **greeting** and **countButton**.

	findViewById	View Binding
Type safety	You choose the type; a wrong guess crashes	The right type is generated
Null safety	Wrong id = null = crash at runtime	Only ids in this layout exist; mistakes don't compile
Code	One line per view	binding.greeting, binding.countButton

2. The standard pattern (memorise it)

HelloFragment.kt with View Binding

```
class HelloBindingFragment : Fragment() {

    private var _binding: FragmentHelloBinding? = null
    // Only valid between onCreateView and onDestroyView.
    private val binding get() = _binding!!

    private var count = 0

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View {
        _binding = FragmentHelloBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        binding.countButton.setOnClickListener {
            count++
            binding.greeting.text = "Tapped $count times"
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null              // the view is gone: let it be garbage-collected
    }
}
```

Line	Why
_binding: ...? = null	Nullable, because the binding only exists while the view exists
binding get() = _binding!!	Convenient non-null access in code that runs while the view exists
inflate(inflater, container, false)	Creates the views; false = don't attach yet, the fragment does that
return binding.root	The root view of the layout becomes the fragment's view
_binding = null in onDestroyView	The fragment may live on (back stack) after its view dies. Keeping the binding would keep the whole old view in memory: a leak

Careful: Using **binding** after onDestroyView crashes with a NullPointerException (from the !!). If you see that crash, some code (a callback, a coroutine, a listener) is running after the view is gone. Tie that work to viewLifecycleOwner.

3. A shorter version with bind()

If the layout is passed to the constructor, you can skip onCreateView and use **bind(view)**:

Shorter: Fragment(R.layout...) + bind()

```
class HelloShortFragment : Fragment(R.layout.fragment_hello) {
    private var _binding: FragmentHelloBinding? = null
    private val binding get() = _binding!!

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentHelloBinding.bind(view)
        binding.greeting.text = "Hello, Anjana"
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Tip: Both versions are common. Use whichever your team uses: consistency matters more than which one you pick.

4. A small form fragment

res/layout/fragment_login.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/emailInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email"
        android:inputType="textEmailAddress" />

    <EditText
        android:id="@+id/passwordInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword" />

    <TextView
        android:id="@+id/errorText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="#C62828"
        android:visibility="gone" />

    <Button
        android:id="@+id/loginButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Log in" />
</LinearLayout>
```

LoginFragment.kt

```
class LoginFragment : Fragment(R.layout.fragment_login) {
    private var _binding: FragmentLoginBinding? = null
    private val binding get() = _binding!!

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentLoginBinding.bind(view)

        binding.loginButton.setOnClickListener {
            val email = binding.emailInput.text.toString().trim()
            val password = binding.passwordInput.text.toString()
            val error = when {
                !email.contains("@") -> "Enter a valid email"
                password.length < 8 -> "Password must be at least 8 characters"
                else -> null
            }
            binding.errorText.text = error ?: ""
            binding.errorText.visibility = if (error == null) View.GONE else View.VISIBLE
            if (error == null) {
                Toast.makeText(requireContext(), "Welcome!", Toast.LENGTH_SHORT).show()
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Visibility	Meaning
View.VISIBLE	Shown
View.INVISIBLE	Hidden, but still takes up its space
View.GONE	Hidden and takes no space

Tip: Putting the validation inside the fragment is fine for learning. In team code the rules go into a **ViewModel** (Day 5), and the fragment only shows the result. The same idea as UiState in Compose.

Practice Exercises

1. Convert your ProfileFragment from Day 1 to View Binding using the onCreateView + inflate pattern.

2. In LoginFragment, disable the login button (**isEnabled = false**) while either field is empty. Hint: **binding.emailInput.doAfterTextChanged { ... }** runs after every change.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What binding class name does activity_settings.xml generate?

Q2. Why is _binding set to null in onDestroyView?

Q3. What happens if you use binding after onDestroyView?

Q4. What's the difference between View.INVISIBLE and View.GONE?

BONUS MODULE • DAY 4

FragmentManager, Arguments and Results

Today's goal: Add, replace and remove fragments, use the back stack, pass data into a fragment with arguments, and send results back.

1. The FragmentManager

Every activity has a **FragmentManager** that keeps track of its fragments and the back stack. You change fragments in a **transaction**: a group of changes applied together.

Where you are	Use	It manages
In an activity	supportFragmentManager	The activity's fragments
In a fragment	parentFragmentManager	The manager this fragment belongs to (to swap itself or siblings)
In a fragment	childFragmentManager	Fragments placed INSIDE this fragment (e.g. tabs)

2. Transactions

Operation	What it does
add<F>(containerId)	Adds a fragment on top of what's already in the container
replace<F>(containerId)	Removes the fragments in the container and adds the new one (most common)
remove(fragment)	Removes a fragment
addToBackStack(name)	Remembers this transaction, so Back undoes it
setReorderingAllowed(true)	Lets the system optimise the transaction. Always add it
setCustomAnimations(...)	Animations for enter, exit, and for Back

Going from the list to a detail screen

```
class NotesListFragment : Fragment(R.layout.fragment_hello) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        view.findViewById<Button>(R.id.countButton).setOnClickListener {
            parentFragmentManager.commit {
                setReorderingAllowed(true)
                setCustomAnimations(
                    android.R.anim.slide_in_left, android.R.anim.fade_out,     // enter, exit
                    android.R.anim.fade_in, android.R.anim.slide_out_right,    // popEnter, popExit (Back)
                )
                replace(R.id.fragmentContainer, NoteDetailFragment.newInstance(noteId = 7, title = "Shopping"))
                addToBackStack("detail")
            }
        }
    }
}
```

Sketch: The back stack

```
Start:              [ NotesList ]
After replace +     [ NotesList ] <- [ NoteDetail ]      (NotesList's view is destroyed,
  addToBackStack                                          the fragment waits)
Press Back:         [ NotesList ]                        (NotesList gets a new view)
Without addToBackStack, Back would leave the app instead.
```

Other back stack helpers

```
fun backStackExamples(fm: FragmentManager) {
    fm.popBackStack()                                          // like pressing Back once
    fm.popBackStack("detail", FragmentManager.POP_BACK_STACK_INCLUSIVE)   // back to before "detail"
    val count = fm.backStackEntryCount                         // how many entries
    val current = fm.findFragmentById(R.id.fragmentContainer)    // the fragment shown now
    Log.d("BackStack", "entries=$count, current=$current")
}
```

3. Passing data in: arguments

To open a detail screen you need to pass the note's id. With fragments you **never** pass data through the constructor. You put it in a **Bundle** called **arguments**.

WRONG: data in the constructor

```
class NoteDetailFragment(val noteId: Int) : Fragment()     // crashes after rotation or process death
```

Android recreates fragments itself (after rotation, or when the app comes back after being killed in the background) using the **empty constructor**. Constructor values are lost, and a fragment without an empty constructor crashes. Arguments, however, are saved and restored automatically.

NoteDetailFragment.kt with newInstance and arguments

```
class NoteDetailFragment : Fragment(R.layout.fragment_note_detail) {

    companion object {
        private const val ARG_NOTE_ID = "note_id"
        private const val ARG_TITLE = "title"

        // The only way other code should create this fragment.
        fun newInstance(noteId: Int, title: String) = NoteDetailFragment().apply {
            arguments = bundleOf(ARG_NOTE_ID to noteId, ARG_TITLE to title)
        }
    }

    private var _binding: FragmentNoteDetailBinding? = null
    private val binding get() = _binding!!

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentNoteDetailBinding.bind(view)

        val noteId = requireArguments().getInt(ARG_NOTE_ID)
        val title = requireArguments().getString(ARG_TITLE) ?: ""
        binding.titleText.text = "#$noteId  $title"
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Piece	Why
companion object { fun newInstance(...) }	One clear place that builds the fragment with correct arguments
bundleOf(key to value)	Creates a Bundle; values can be Int, String, Boolean, Long, Parcelable...
requireArguments()	Returns the arguments, or crashes with a clear message if someone forgot them
const val ARG_... keys	No typos between writing and reading

Tip: With the **replace(containerId, Class, args)** form you can also write **replace<NoteDetailFragment>(R.id.fragmentContainer, args = bundleOf(...))**. On Day 6, **Safe Args** generates all of this for you.

4. Sending a result back: the Fragment Result API

Sometimes a screen returns something: a picked colour, a confirmed choice. Fragments don't call each other directly (they might not even exist at the same time). They use the **Fragment Result API**: one side listens for a **request key**, the other side sets a result with that key.

Picker sends, list receives

```
// The screen that picks a colour and goes back
class ColorPickerFragment : Fragment(R.layout.fragment_hello) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        view.findViewById<Button>(R.id.countButton).setOnClickListener {
            setFragmentResult("pick_color", bundleOf("color" to "Green"))
            parentFragmentManager.popBackStack()
        }
    }
}

// The screen that opened the picker and waits for the answer
class SettingsFragment : Fragment(R.layout.fragment_hello) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setFragmentResultListener("pick_color") { _, bundle ->
            val color = bundle.getString("color")
            Toast.makeText(requireContext(), "You picked $color", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Rule	Why
Both sides use the same key ("pick_color")	The key connects sender and receiver
Register the listener in onCreate	It must exist before the result arrives; it survives the receiver's view being destroyed
The result is delivered when the receiver is STARTED	So it's safe to update the UI or show a Toast

Practice Exercises

1. Create **ProductFragment.newInstance(name: String, price: Int)** that shows "<name> - Rs. <price>" from its arguments. Open it from a list fragment with replace + addToBackStack.

2. Create a **RatingFragment** with three buttons (1, 2, 3 stars). Tapping one sends the rating back with setFragmentResult("rating", ...) and pops the back stack. The opening fragment shows "Rated 2 stars".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What's the difference between add and replace?

Q2. What does addToBackStack do?

Q3. Why can't you pass data through a fragment's constructor?

Q4. When should you use childFragmentManager?

BONUS MODULE • DAY 5

ViewModel, Flow and RecyclerView in Fragments

Today's goal: Build a real list screen: a ViewModel with StateFlow, a RecyclerView with ListAdapter, and lifecycle-safe Flow collection.

1. The team pattern

Sketch: How a fragment screen is organised

```
NotesFragment (UI only)                NotesViewModel (logic + state)
  - inflates the layout       events   - StateFlow<NotesUiState>
  - sets listeners          ------->   - toggleDone(), add()
  - collects uiState        <-------   - survives rotation
  - submits the list to       state
    the adapter
```

It's exactly the same idea as MVVM in Compose (Week 8): the ViewModel owns the state, the screen just shows it and reports what the user did. Only the way of showing it changes: **views and an adapter** instead of composables.

2. The ViewModel

NotesViewModel.kt

```
data class Note(val id: Int, val title: String, val isDone: Boolean = false)

data class NotesUiState(
    val isLoading: Boolean = true,
    val notes: List<Note> = emptyList(),
)

class NotesViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(NotesUiState())
    val uiState: StateFlow<NotesUiState> = _uiState.asStateFlow()

    init {
        viewModelScope.launch {
            delay(500)                              // pretend to load from a database
            _uiState.value = NotesUiState(isLoading = false, notes = listOf(
                Note(1, "Learn fragments"), Note(2, "Practise View Binding"), Note(3, "Read team code"),
            ))
        }
    }

    fun toggleDone(id: Int) = _uiState.update { state ->
        state.copy(notes = state.notes.map { if (it.id == id) it.copy(isDone = !it.isDone) else it })
    }

    fun add(title: String) = _uiState.update { state ->
        val nextId = (state.notes.maxOfOrNull { it.id } ?: 0) + 1
        state.copy(notes = state.notes + Note(nextId, title))
    }
}
```

3. The layouts

res/layout/fragment_notes.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <EditText
            android:id="@+id/titleInput"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="New note" />
        <Button
            android:id="@+id/addButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Add" />
    </LinearLayout>

    <ProgressBar
        android:id="@+id/progress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/notesList"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
```

res/layout/item_note.xml (one row)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="12dp">
    <CheckBox
        android:id="@+id/noteDone"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:clickable="false" />
    <TextView
        android:id="@+id/noteTitle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_gravity="center_vertical"
        android:textSize="18sp" />
</LinearLayout>
```

4. The adapter: RecyclerView + ListAdapter

A **RecyclerView** shows long lists efficiently by reusing row views. It's the View-world version of LazyColumn. **ListAdapter** with **DiffUtil** works out which rows changed when you submit a new list, and animates only those.

NotesAdapter.kt

```
class NotesAdapter(
    private val onClick: (Note) -> Unit,
) : ListAdapter<Note, NotesAdapter.NoteViewHolder>(NoteDiff) {

    class NoteViewHolder(val binding: ItemNoteBinding) : RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): NoteViewHolder {
        val binding = ItemNoteBinding.inflate(LayoutInflater.from(parent.context), parent, false)
        return NoteViewHolder(binding)
    }

    override fun onBindViewHolder(holder: NoteViewHolder, position: Int) {
        val note = getItem(position)
        holder.binding.noteTitle.text = note.title
        holder.binding.noteDone.isChecked = note.isDone
        holder.binding.root.setOnClickListener { onClick(note) }
    }

    object NoteDiff : DiffUtil.ItemCallback<Note>() {
        override fun areItemsTheSame(oldItem: Note, newItem: Note) = oldItem.id == newItem.id
        override fun areContentsTheSame(oldItem: Note, newItem: Note) = oldItem == newItem
    }
}
```

Piece	Role
ViewHolder	Holds one row's views (here the row's binding)
onCreateViewHolder	Creates a new row view (called only a few times; rows are reused)
onBindViewHolder	Fills a row with the data at a position (called a lot, keep it fast)
areItemsTheSame	Same item? Compare ids
areContentsTheSame	Did anything shown change? Compare the whole data class
submitList(list)	Give the adapter a new list; DiffUtil updates only what changed

5. The fragment

NotesFragment.kt

```
class NotesFragment : Fragment(R.layout.fragment_notes) {

    private val viewModel: NotesViewModel by viewModels()     // survives rotation

    private var _binding: FragmentNotesBinding? = null
    private val binding get() = _binding!!

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentNotesBinding.bind(view)

        val adapter = NotesAdapter(onClick = { note -> viewModel.toggleDone(note.id) })
        binding.notesList.layoutManager = LinearLayoutManager(requireContext())
        binding.notesList.adapter = adapter

        binding.addButton.setOnClickListener {
            val title = binding.titleInput.text.toString().trim()
            if (title.isNotEmpty()) {
                viewModel.add(title)
                binding.titleInput.setText("")
            }
        }

        // Collect the state only while the view is at least STARTED.
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect { state ->
                    binding.progress.visibility = if (state.isLoading) View.VISIBLE else View.GONE
                    adapter.submitList(state.notes)
                }
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        binding.notesList.adapter = null      // the RecyclerView would otherwise keep the adapter
        _binding = null
    }
}
```

Line	Why
by viewModels()	Gets the ViewModel for this fragment; the same instance comes back after rotation
viewLifecycleOwner.lifecycleScope	The coroutine is cancelled when the view is destroyed (onDestroyView)
repeatOnLifecycle(STARTED)	Collects only while visible; stops in the background and restarts when the user returns
adapter.submitList(state.notes)	The adapter updates the rows that changed
notesList.adapter = null	Avoids the RecyclerView keeping the adapter (and the old view) alive

Careful: Don't write **lifecycleScope.launch { viewModel.uiState.collect {...} }** in a fragment (without viewLifecycleOwner and repeatOnLifecycle). It keeps collecting in the background and after the view is destroyed, which wastes battery and crashes when binding is null.

6. Sharing data between fragments

Two fragments in the same activity can share one ViewModel with **activityViewModels()**. For example, a filter screen and a list screen.

A shared ViewModel

```
class FilterViewModel : ViewModel() {
    private val _onlyOpen = MutableStateFlow(false)
    val onlyOpen: StateFlow<Boolean> = _onlyOpen.asStateFlow()
    fun setOnlyOpen(value: Boolean) { _onlyOpen.value = value }
}

class FilterFragment : Fragment(R.layout.fragment_hello) {
    private val filterViewModel: FilterViewModel by activityViewModels()     // the activity's instance
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        view.findViewById<Button>(R.id.countButton).setOnClickListener { filterViewModel.setOnlyOpen(true) }
    }
}

class FilteredListFragment : Fragment(R.layout.fragment_hello) {
    private val filterViewModel: FilterViewModel by activityViewModels()     // the SAME instance
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val label = view.findViewById<TextView>(R.id.greeting)
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                filterViewModel.onlyOpen.collect { label.text = if (it) "Open notes only" else "All notes" }
            }
        }
    }
}
```

Delegate	Scope	Use for
by viewModels()	This fragment	The screen's own state (most common)
by activityViewModels()	The host activity	State shared by several fragments
by viewModels({ requireParentFragment() })	The parent fragment	Sharing between child fragments (e.g. tabs)

7. With Hilt

Fragments with Hilt (Week 12)

```
class NotesRepository @Inject constructor()

@HiltViewModel
class HiltNotesViewModel @Inject constructor(
    private val repository: NotesRepository,
) : ViewModel()

@AndroidEntryPoint                      // needed on every fragment that uses Hilt...
class HiltNotesFragment : Fragment(R.layout.fragment_notes) {
    private val viewModel: HiltNotesViewModel by viewModels()    // ...and the same by viewModels() works
}
```

Tip: If a fragment is @AndroidEntryPoint, its host activity must be @AndroidEntryPoint too, or the app crashes with "Hilt Fragments must be attached to an @AndroidEntryPoint Activity".

Practice Exercises

1. Add a **delete** feature: a long press on a row (**setOnLongClickListener { ...; true }**) removes the note. Add **delete(id)** to the ViewModel.

2. Show a TextView "No notes yet" (id **emptyText**) when the list is empty and loading has finished. Set its visibility in the collect block.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why collect Flows with viewLifecycleOwner.lifecycleScope and repeatOnLifecycle(STARTED)?

Q2. What does DiffUtil do for ListAdapter?

Q3. What's the difference between viewModels() and activityViewModels()?

Q4. Which RecyclerView adapter method is called most often?

BONUS MODULE • DAY 6

Navigation, Dialogs and Compose in Fragments

Today's goal: Use the Navigation component with a nav graph and Safe Args, add bottom navigation and dialogs, and put Compose inside a fragment.

1. The Navigation component for fragments

Writing transactions by hand (Day 4) gets messy in a big app. Most teams use the **Navigation component**: you draw screens and arrows in a **navigation graph**, and the library does the transactions and the back stack.

Setup (Android only, versions as in your project)

```
// build.gradle.kts (Project)
plugins {
    id("androidx.navigation.safeargs.kotlin") version "2.8.4" apply false
}

// app/build.gradle.kts
plugins {
    id("androidx.navigation.safeargs.kotlin")       // generates Directions and Args classes
}
dependencies {
    implementation("androidx.navigation:navigation-fragment-ktx:2.8.4")
    implementation("androidx.navigation:navigation-ui-ktx:2.8.4")
}
```

res/navigation/nav_graph.xml

```
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/notesFragment">

    <fragment
        android:id="@+id/notesFragment"
        android:name="com.anjana.fragments.NotesFragment"
        android:label="Notes">
        <action
            android:id="@+id/action_notes_to_detail"
            app:destination="@id/noteDetailFragment" />
    </fragment>

    <fragment
        android:id="@+id/noteDetailFragment"
        android:name="com.anjana.fragments.NoteDetailFragment"
        android:label="Note">
        <argument android:name="noteId" app:argType="integer" />
        <argument android:name="title" app:argType="string" />
    </fragment>
</navigation>
```

activity_main.xml hosts a NavHostFragment

```
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/navHost"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph" />
```

XML	Meaning
<fragment android:id=... android:name=...>	A destination (a screen) and its class
<action app:destination=...>	An arrow: a way to go from this screen to another
<argument app:argType=...>	Data the destination needs (integer, string, boolean, long, Parcelable...)
app:defaultNavHost="true"	The system Back button goes back in the graph

Tip: Android Studio has a visual **Design** tab for nav_graph.xml where you can drag screens and draw arrows. It writes this XML for you.

2. Navigating with Safe Args

Safe Args **generates** a class for every action (**NotesFragmentDirections**) and for every destination's arguments (**NoteDetailFragmentArgs**), so wrong names or types don't compile.

Sending and receiving with Safe Args

```
class NavNotesFragment : Fragment(R.layout.fragment_hello) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        view.findViewById<Button>(R.id.countButton).setOnClickListener {
            val action = NotesFragmentDirections.actionNotesToDetail(noteId = 7, title = "Shopping")
            findNavController().navigate(action)
        }
    }
}

class NavNoteDetailFragment : Fragment(R.layout.fragment_note_detail) {
    private val args: NoteDetailFragmentArgs by navArgs()       // reads the arguments for you

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val binding = FragmentNoteDetailBinding.bind(view)
        binding.titleText.text = "#${args.noteId}  ${args.title}"
        binding.backButton.setOnClickListener { findNavController().navigateUp() }
    }
}
```

Day 4 (by hand)	Navigation component
parentFragmentManager.commit { replace(...); addToBackStack(...) }	findNavController().navigate(action)
newInstance() + bundleOf + requireArguments()	Generated Directions + by navArgs()
popBackStack()	findNavController().navigateUp() or popBackStack()

In a ViewModel you can read the same arguments from **SavedStateHandle**: **savedStateHandle.get<Int>("noteId")**, so the ViewModel can load the note itself.

3. Bottom navigation

res/menu/bottom_nav.xml (item ids = destination ids in the graph)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/notesFragment" android:icon="@drawable/ic_notes" android:title="Notes" />
    <item android:id="@+id/settingsFragment" android:icon="@drawable/ic_settings" android:title="Settings" />
</menu>
```

Connecting the BottomNavigationView

```
class MainNavActivity : AppCompatActivity(R.layout.activity_main) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val navHost = supportFragmentManager.findFragmentById(R.id.navHost) as NavHostFragment
        val navController = navHost.navController
        findViewById<BottomNavigationView>(R.id.bottomNav).setupWithNavController(navController)
    }
}
```

Careful: The menu item ids must be **exactly the same** as the destination ids in nav_graph.xml. If they differ, tapping a tab does nothing. That's the first thing to check when bottom navigation "doesn't work".

4. Dialogs: DialogFragment and bottom sheets

Use a **DialogFragment** for dialogs, not a plain AlertDialog created in a click listener. A DialogFragment survives rotation; a plain dialog disappears (and can leak).

A confirm dialog that sends a result

```
class ConfirmDeleteDialog : DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog =
        AlertDialog.Builder(requireContext())
            .setTitle("Delete note?")
            .setMessage("This can't be undone.")
            .setPositiveButton("Delete") { _, _ -> setFragmentResult("confirm_delete", bundleOf("ok" to true)) }
            .setNegativeButton("Cancel", null)
            .create()
}

class NoteActionsSheet : BottomSheetDialogFragment() {        // slides up from the bottom
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View =
        inflater.inflate(R.layout.fragment_hello, container, false)
}

fun showDialogs(fragment: Fragment) {
    ConfirmDeleteDialog().show(fragment.childFragmentManager, "confirm_delete")
    NoteActionsSheet().show(fragment.childFragmentManager, "note_actions")
}
```

5. Compose inside a fragment

Teams moving to Compose usually do it **one screen at a time**: the fragment stays (so navigation doesn't change) but its UI becomes Compose. You'll see this a lot.

A fragment whose UI is Compose

```
class ComposeNotesFragment : Fragment() {
    private val viewModel: NotesViewModel by viewModels()

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View =
        ComposeView(requireContext()).apply {
            // Dispose the composition when the fragment's VIEW is destroyed
            setViewCompositionStrategy(ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed)
            setContent {
                MaterialTheme {
                    val state by viewModel.uiState.collectAsState()
                    LazyColumn(Modifier.fillMaxSize().padding(16.dp)) {
                        items(state.notes, key = { it.id }) { note ->
                            Text(
                                text = (if (note.isDone) "[x] " else "[ ] ") + note.title,
                                modifier = Modifier.fillMaxWidth().clickable { viewModel.toggleDone(note.id) }.padding(12.dp),
                            )
                        }
                    }
                }
            }
        }
}
```

Piece	Why
ComposeView(requireContext())	A normal View that can show composables
setViewCompositionStrategy(DisposeOnViewTreeLifecycleDestroyed)	Ties the composition to the fragment's view lifecycle, matching onDestroyView
Same NotesViewModel as Day 5	Only the UI changed: that's why keeping logic in the ViewModel pays off

Tip: In real projects use **collectAsStateWithLifecycle()** (lifecycle-runtime-compose) instead of collectAsState(). It stops collecting in the background, like repeatOnLifecycle.

6. Common fragment errors and their fixes

Error / symptom	Usual cause	Fix
Fragment not attached to a context	requireContext() called after the user left	Do the work in viewLifecycleOwner.lifecycleScope
NullPointerException at binding	Using binding after onDestroyView	Cancel callbacks with the view lifecycle
Fragment duplicated after rotation	Adding in onCreate without the savedInstanceState check	if (savedInstanceState == null)
Unable to instantiate fragment ... empty constructor	Constructor with parameters	Use arguments / newInstance / Safe Args
Can not perform this action after onSaveInstanceState	commit() after the activity was stopped	Commit in response to user actions; in rare cases commitAllowingStateLoss()
navigation destination ... is unknown to this NavController	Navigating twice fast, or from the wrong screen	Disable the button after one tap; check the current destination
Data shows twice / UI updates twice	Collecting with the fragment's lifecycle	Use viewLifecycleOwner + repeatOnLifecycle

7. Mini project: Notes with fragments

- **NotesFragment**: the Day 5 list (ViewModel + RecyclerView). Tapping a row navigates with Safe Args to the detail.
- **NoteDetailFragment**: shows the note from its arguments, with a Delete button that opens **ConfirmDeleteDialog**.

- When the dialog confirms, the detail screen tells the shared ViewModel (activityViewModels) to delete and navigates up.
- **SettingsFragment** in a bottom navigation bar, with a switch that hides done notes (shared FilterViewModel).
- Bonus: rewrite NoteDetailFragment's UI with ComposeView, keeping everything else the same.

Practice Exercises

1. Add an **aboutFragment** destination and an action to it from notesFragment. Open it from a menu button with findNavController().navigate(R.id.action_notes_to_about).

2. In NavNoteDetailFragment, add a Delete button that shows ConfirmDeleteDialog, and listen for the "confirm_delete" result with **childFragmentManager.setFragmentResultListener(...)**. When it arrives, show a Toast and navigate up.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does Safe Args generate?

Q2. Why use DialogFragment instead of creating an AlertDialog directly?

Q3. Why must bottom menu item ids match the nav graph destination ids?

Q4. Which ViewCompositionStrategy fits a ComposeView in a fragment?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: What Is a Fragment?

Exercise 1

```
class ProfileFragment : Fragment(R.layout.fragment_profile) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val followButton = view.findViewById<Button>(R.id.followButton)
        followButton.setOnClickListener {
            followButton.text = "Following"
        }
    }
}

// In MainActivity.onCreate:
// if (savedInstanceState == null) {
//     supportFragmentManager.commit { setReorderingAllowed(true); add<ProfileFragment>(R.id.fragmentContainer) }
// }
```

Exercise 2

```
class HelloNameFragment : Fragment(R.layout.fragment_hello_name) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val nameInput = view.findViewById<EditText>(R.id.nameInput)
        val greeting = view.findViewById<TextView>(R.id.greeting)
        view.findViewById<Button>(R.id.countButton).setOnClickListener {
            val name = nameInput.text.toString().trim()
            if (name.isEmpty()) {
                Toast.makeText(requireContext(), "Enter your name", Toast.LENGTH_SHORT).show()
            } else {
                greeting.text = "Hello, $name!"
            }
        }
    }
}
```

Quiz answers

Q1. No. A fragment always lives inside a host activity.

Q2. A view in the activity's layout where fragments are shown.

Q3. After rotation Android restores the fragments itself; adding again would create duplicates.

Q4. onViewCreated.

Day 2: The Fragment Lifecycle

Exercise 1

```
// Rotating twice: onCreate runs 3 times in total (once at start + once per rotation),
// because the fragment object is recreated every time.
class ProfileLoggingFragment : Fragment(R.layout.fragment_profile) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("Lifecycle", "onCreate, restored=${savedInstanceState != null}")
    }
}
```

Exercise 2

```
class CountingFragment : Fragment(R.layout.fragment_hello) {
    private var count = 0

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        count = savedInstanceState?.getInt("count") ?: 0
        val greeting = view.findViewById<TextView>(R.id.greeting)
        greeting.text = "Tapped $count times"
        view.findViewById<Button>(R.id.countButton).setOnClickListener {
            count++
            greeting.text = "Tapped $count times"
        }
    }

    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        outState.putInt("count", count)
    }
}
```

Quiz answers

Q1. onDestroyView (onDestroy does not run).

Q2. The lifecycle of the fragment's view: from onViewCreated to onDestroyView.

Q3. The view can be destroyed and recreated while the fragment lives; observing with the fragment's lifecycle leaks and duplicates observers.

Q4. requireContext() returns a non-null Context or crashes; context returns null when not attached.

Day 3: View Binding in Fragments

Exercise 1

```
class ProfileBindingFragment : Fragment() {
    private var _binding: FragmentProfileBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View {
        _binding = FragmentProfileBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        binding.followButton.setOnClickListener { binding.followButton.text = "Following" }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Exercise 2

```
// Inside LoginFragment.onViewCreated, after _binding = FragmentLoginBinding.bind(view):
fun setUpLoginButton(binding: FragmentLoginBinding) {
    fun updateButton() {
        binding.loginButton.isEnabled =
            binding.emailInput.text.isNotBlank() && binding.passwordInput.text.isNotBlank()
    }
    binding.emailInput.doAfterTextChanged { updateButton() }
    binding.passwordInput.doAfterTextChanged { updateButton() }
    updateButton()                          // correct state at the start
}
```

Quiz answers

Q1. ActivitySettingsBinding.

Q2. The fragment can outlive its view (back stack); holding the binding would keep the old views in memory.

Q3. NullPointerException from _binding!!.

Q4. INVISIBLE keeps the space; GONE removes it from the layout.

Day 4: FragmentManager, Arguments and Results

Exercise 1

```
class ProductFragment : Fragment(R.layout.fragment_note_detail) {
    companion object {
        private const val ARG_NAME = "name"
        private const val ARG_PRICE = "price"
        fun newInstance(name: String, price: Int) = ProductFragment().apply {
            arguments = bundleOf(ARG_NAME to name, ARG_PRICE to price)
        }
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val name = requireArguments().getString(ARG_NAME) ?: ""
        val price = requireArguments().getInt(ARG_PRICE)
        FragmentNoteDetailBinding.bind(view).titleText.text = "$name - Rs. $price"
    }
}

// In the list fragment:
fun openProduct(fragment: Fragment) {
    fragment.parentFragmentManager.commit {
        setReorderingAllowed(true)
        replace(R.id.fragmentContainer, ProductFragment.newInstance("Notebook", 60))
        addToBackStack(null)
    }
}
```

Exercise 2

```
class RatingFragment : Fragment(R.layout.fragment_rating) {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val buttons = listOf(R.id.oneStar to 1, R.id.twoStars to 2, R.id.threeStars to 3)
        for ((id, stars) in buttons) {
            view.findViewById<Button>(id).setOnClickListener {
                setFragmentResult("rating", bundleOf("stars" to stars))
                parentFragmentManager.popBackStack()
            }
        }
    }
}

class ReviewFragment : Fragment(R.layout.fragment_hello) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setFragmentResultListener("rating") { _, bundle ->
            val stars = bundle.getInt("stars")
            view?.findViewById<TextView>(R.id.greeting)?.text = "Rated $stars stars"
        }
    }
}
```

Quiz answers

Q1. add puts a fragment on top of existing ones; replace removes the existing fragments in the container first.

Q2. Saves the transaction so that pressing Back reverses it.

Q3. Android recreates fragments with the empty constructor (rotation, process death), so the data would be lost.

Q4. For fragments shown inside another fragment, like tabs inside a screen.

Day 5: ViewModel, Flow and RecyclerView in Fragments

Exercise 1

```
// NotesViewModel:
fun delete(id: Int) = _uiState.update { state -> state.copy(notes = state.notes.filterNot { it.id == id }) }

// NotesAdapter: add a second callback, onLongClick: (Note) -> Unit, and in onBindViewHolder:
// holder.binding.root.setOnLongClickListener { onLongClick(note); true }   // true = "handled"

// NotesFragment:
// val adapter = NotesAdapter(onClick = { viewModel.toggleDone(it.id) }, onLongClick = { viewModel.delete(it.id) })
```

Exercise 2

```
// Inside the collect block of NotesFragment:
fun showState(binding: FragmentNotesBinding, state: NotesUiState) {
    binding.progress.visibility = if (state.isLoading) View.VISIBLE else View.GONE
    binding.emptyText.visibility =
        if (!state.isLoading && state.notes.isEmpty()) View.VISIBLE else View.GONE
}
```

Quiz answers

Q1. So collection stops when the view is destroyed and pauses in the background; no leaks or crashes on a null binding.

Q2. It compares the old and new lists and updates only the rows that changed.

Q3. viewModels() is scoped to the fragment; activityViewModels() gives the activity's instance, shared by its fragments.

Q4. onBindViewHolder.

Day 6: Navigation, Dialogs and Compose in Fragments

Exercise 1

```
// nav_graph.xml, inside <fragment android:id="@+id/notesFragment" ...>:
//     <action android:id="@+id/action_notes_to_about" app:destination="@id/aboutFragment" />
// and a new destination:
//     <fragment android:id="@+id/aboutFragment" android:name="com.anjana.fragments.AboutFragment" android:label="About" />

class AboutFragment : Fragment(R.layout.fragment_hello)

fun openAbout(fragment: Fragment) {
    fragment.findNavController().navigate(R.id.action_notes_to_about)
}
```

Exercise 2

```
class DeletableDetailFragment : Fragment(R.layout.fragment_note_detail) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        childFragmentManager.setFragmentResultListener("confirm_delete", this) { _, bundle ->
            if (bundle.getBoolean("ok")) {
                Toast.makeText(requireContext(), "Deleted", Toast.LENGTH_SHORT).show()
                findNavController().navigateUp()
            }
        }
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        FragmentNoteDetailBinding.bind(view).deleteButton.setOnClickListener {
            ConfirmDeleteDialog().show(childFragmentManager, "confirm_delete")
        }
    }
}
```

Quiz answers

Q1. Directions classes for actions and Args classes for destination arguments, so navigation is type-safe.

Q2. A DialogFragment is managed by the FragmentManager, so it survives rotation and doesn't leak.

Q3. setupWithNavController navigates to the destination with the same id as the tapped item.

Q4. DisposeOnViewTreeLifecycleDestroyed, so the composition ends with the fragment's view.

Next: Open your team's project and find three fragments. For each one, name its layout, how it gets its data (arguments, ViewModel), and how it navigates. If you can do that, you understand fragments. Well done, Anjana!